

# Cognitive Cartography

A Meta-Methodology for Human–AI Collaboration on Complex Problems

Oleg Roshka\*

April 2026

## Abstract

Modern AI agents have characteristic limitations on long-horizon complex work: bounded context windows, drift across iterations, phantom decisions (claims of resolution not grounded in any artefact), reference rot, lost intent, and the cost of restarting cold each session. These limits will not be dissolved by larger models — they are limits of *substrate*, the externalised state any sustained reasoning must traverse. We articulate **Cognitive Cartography**: a meta-methodology the author has practised across many domains and many years of sustained complex work, made explicit here in response to the demands of multi-session AI collaboration. Cognitive Cartography names a discipline a person cultivates, with or without AI as participant, of forming, maintaining, and cascading hierarchical external representations across the manifestation trajectory of a project. The discipline rests on long-standing cognitive-science foundations — distributed cognition (Hutchins, 1995), external representations (Scaife and Rogers, 1996), cognitive load (Sweller, 1988), hierarchical levels of analysis (Marr, 1982), boundary objects (Star and Griesemer, 1989) — and recovers software-engineering precedent (Architectural Decision Records (Nygard, 2011), Single Source of Truth, hexagonal architecture (Cockburn, 2005), append-only logs (Candea and Fox, 2003)) alongside emerging agentic-AI patterns (Lewis et al., 2020; Packer et al., 2024; Shinn et al., 2023; Bai et al., 2022). The discipline defines six artefact categories — Knowledge Bases, Inventories, Data Dictionaries, Decision Records, Open Questions, Glossary — a status lifecycle, an edit protocol with explicit anti-patterns and a trivial-fix lane, and forward/reverse propagation rules across an explicit hierarchy of abstraction layers. Each element maps to a cognitive principle and to a specific failure mode it mitigates. The practice is domain-agnostic. It converts chat-based AI collaboration into a durable, multi-session, multi-abstraction discipline. We claim the substrate matters as much as the model, the discipline is practicable today by anyone with no special tooling, and that this is the form of work AI raises in importance — not invents.

## 1 Why Substrate Matters as Much as Model

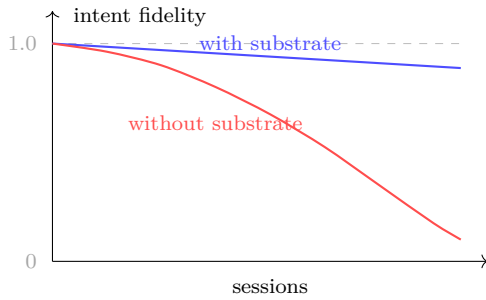
Most discussions of AI capability concern the model: its parameters, its benchmarks, the length of its context window. Almost none concern the *substrate* — the externalised state any sustained reasoning, whether organic or artificial, must traverse. This paper is about the substrate.

The thesis is simple to state and difficult to live up to. **For complex tasks that span many sessions, many decisions, and many abstraction layers, the substrate matters at least as much as the model.** A larger model with no substrate degrades along predictable axes; a moderate model with a well-tended substrate can sustain coherence over arbitrary horizons. Bigger context windows are a quantitative response to a qualitative problem. They postpone the failure

modes; they do not dissolve them.

By *substrate* we mean the persistent, externally visible state of the work — the artefacts read at the start of a session, written to during it, and left behind for the next. Informally, the substrate is whatever remnants survive between conversations: a chat transcript, scattered notes, the source code itself. Disciplined, it is something else — a structured, hierarchical, multi-artefact graph that encodes the *current state* of the project’s understanding, with explicit semantics for each kind of artefact and explicit rules for keeping the graph coherent. The remainder of this paper concerns that disciplined practice.

We call it **Cognitive Cartography**. The metaphor is deliberate. Cartography is the art and science of map-making, distinct from the territory it represents. A good map sustains the traveller across many crossings, supports naviga-



**Figure 1:** The drift spectrum. Without substrate discipline, intent fidelity decays superlinearly across sessions; with substrate, decay is bounded.

tion at multiple scales, can be updated by successive surveyors without losing fidelity, and uses conventions stable enough that someone returning years later can still read it. The artefacts of Cognitive Cartography are maps in this sense: they shadow the cognitive territory of a project at lower fidelity than full understanding but with substantially higher *stability*. Stability is what sustained work requires — and what the unaided memory of any single mind, organic or artificial, eventually fails to provide (Figure 1).

Two longer traditions deserve immediate credit. The cognitive-science one has known for decades that cognition is not contained in heads — that it is *distributed* across people, tools, and the environment. Hutchins’ analysis of ship navigation (Hutchins, 1995) showed that the cognitive work of plotting a course is performed by a system of people and instruments, not by any one mind. Scaife and Rogers (1996) later distinguished between *internal* representations (in the head) and *external* representations (in the environment), showing how the latter can radically reduce the cognitive load of complex tasks. The software-engineering tradition has, for at least fifteen years, externalised architectural decisions into append-only log entries (Nygard, 2011) precisely to avoid the failure modes catalogued in §2. Cognitive Cartography draws from both. What it adds is the systematic application to a case the older traditions could not have anticipated: where one of the collaborators is a large language model whose every session begins, in cognitive terms, with amnesia.

What follows is shaped by that observation. AI’s most discussed limitations on long-horizon work — drift, phantom decisions, restart cost — are best understood as substrate failures rather than model failures, and respond to substrate-level interventions far more reliably than to model-

level scaling. The discipline that addresses them is not new; it is recovered, made explicit, and applied. And the discipline is fundamentally human: something a person cultivates over time, with AI as participant. AI’s limitations motivate the practice. Cognitive science legitimises it. Practice executes it. This matters, because it means the discipline survives across model generations: it is about how minds collaborate on complex problems, not about which model is current.

The structure of what follows is straightforward. §2 catalogues the failure modes the discipline addresses. §3 traces the foundations from which it draws. §4 specifies the discipline itself. §5 maps each of its elements to the cognitive principle that grounds it and the failure mode it mitigates. §6 shows the discipline in flight on a complex multi-month worked example. §7–§9 discuss cost, tooling, and the honest limits of the practice. §10 closes.

**Provenance.** The discipline articulated here was not invented for this paper, nor co-developed with an AI agent in the writing of it. It is a long-running reflection of the author across many domains and many years — a habit of thought and work shaped by experience well outside the AI context, applied to research, to engineering, and to sustained intellectual work of several kinds. Books, lab notebooks, scientific notation, version control, architectural decision records: each generation finds apparatus for stabilising thought across time and distributing it across minds, and the author has cultivated such apparatus, in one form or another, on every complex project worth sustaining. What working with current-generation language models on complex, multi-session projects did was sharpen the apparatus into something articulable: every implicit affordance of human-only work became an explicit artefact when the collaborator was an agent without persistent memory, and the elements of the discipline became visible in a way they had not been before. This paper names what was already there. The discipline applies wherever sustained complex work crosses many sessions, decisions, or abstraction layers — not only where one of the collaborators is an AI; collaboration with AI is what most recently made the discipline’s elements demand explicit articulation, and gave us the occasion to name them. Acknowledgements at the end credit the AI’s specific role in helping draft and stress-test that articulation.

## 2 What We Observe When Substrate is Absent

The motivating phenomena are not theoretical. They are what one comes to see when collaborating with a large language model on any sufficiently complex task without substrate discipline. The names below are ours; the phenomena are widely reported across the practitioner community and increasingly studied formally (Liu et al., 2024; Park et al., 2023; Packer et al., 2024). Each is described from the inside — what one sees, what one feels, what fails.

### 2.1 Context-window saturation

Every session is bounded by a context window. Even with windows in the hundreds of thousands or millions of tokens, the working set of a long project — accumulated decisions, terminology, partial designs, prior versions — outgrows the window faster than capacity grows. The symptom is a creeping reluctance to introduce new context; a sense of crowding; an instinct to compress prior content into ever-shorter summaries. Each compression discards material the next sub-task might have needed. Capacity addresses the *amount* of state attended to at once, not the *amount* the project has accumulated. These are different quantities, and only the second grows superlinearly with project complexity.

### 2.2 Lost-in-the-middle and attention dilution

When relevant content does fit in the window, it is not equally accessible across positions. The literature on long-context retrieval consistently finds that information placed in the middle of a long context is retrieved less reliably than information at the start or end (Liu et al., 2024). Felt from the human side, this is inconsistent recall: a fact stated three messages ago is honoured; a fact stated thirty messages ago is forgotten or contradicted, even though it remains technically present. Compounding this, every additional irrelevant token dilutes attention that could have been spent on the relevant ones. The agent is the same; the substrate is now noisier.

### 2.3 Drift across iterations

Without an external reference for the project’s settled state, each iteration is built atop the last

iteration’s reconstruction of it. Small reconstruction errors compound. After enough iterations, the running understanding has drifted measurably from any historical anchor. The phenomenon shows up as a sense of “we already decided that, didn’t we?” with the answer increasingly uncertain; or as a drift in terminology, where a term used precisely in week one is used loosely by week three; or as the gentle reinvention of solutions previously rejected for reasons no longer remembered.

### 2.4 Phantom decisions

The most consequential failure mode. Asked “did we decide X?”, the agent answers based on a probabilistic reconstruction from available context — it cannot reliably distinguish “this was decided” from “this was discussed and was a plausible thing to decide”. Without an external decision register, the agent *will confidently confabulate*, citing reasoning that never occurred and conclusions never reached. The human side often discovers this only when the phantom decision contradicts a real one; sometimes only when implementation reveals the inconsistency. The risk is most acute precisely on the decisions that matter most — load-bearing choices made early and referred to often.

### 2.5 Reference rot

In informal practice, references to prior work are by file path or paraphrase: “in the requirements doc, the section about latency”. Reorganise the file or refactor the section, and every such reference becomes stale. The agent, asked to “see the latency section”, produces a plausible reconstruction of what such a section *would have said*. The human side cannot easily distinguish reconstructed content from quoted content. References that were meaningful in week one are increasingly noise by week three.

### 2.6 Restart cost

Every new session begins with whatever substrate can be mustered from memory and immediately-available files. Without a stable warm-up substrate, the early minutes of every session go into re-explaining context: who is doing what, what is being built, what was decided last week, what the open questions are. The cost is borne on one side as fatigue and on the other as crowded context. In aggregate, a project that requires twenty sessions pays the restart cost twenty times.

## 2.7 Decided versus considered confusion

In a chat transcript there is no intrinsic distinction between “this was decided” and “this was considered”. Both look like prose. The human side relies on memory or implicit context to distinguish them. The agent, lacking that mental annotation, treats both as equivalent input. Over time, *considered* options accumulate in the running context with the same status as *decided* ones, and the agent begins to act as if they were settled. Contraries emerge silently, often resolved by the next iteration’s reconstruction in whichever direction the immediate context happens to favour.

### A common shape

These seven failure modes share a single shape. None is a property of any particular model; each is a property of *unsubstantiated collaboration*. Bigger models with longer context windows shift the magnitude of each — the threshold at which it appears, the rate at which it compounds — without changing the underlying topology. Each admits a substrate-level intervention more reliable than any model-level fix. The remainder of the paper specifies those interventions and grounds them in the cognitive-science and software-engineering traditions that have, in different idioms, addressed precisely these failure modes for decades.

## 3 What We Inherit: Foundations

The discipline did not emerge from a vacuum. Three traditions — none of which uses the term *Cognitive Cartography* — converge on its elements. The cognitive-science tradition supplies the deepest framing; the software-engineering tradition supplies hard-won precedent; the agentic-AI literature supplies the contemporary motivation and a small but growing apparatus.

### 3.1 Cognitive science

What Cognitive Cartography asks of a practitioner is not foreign to humans — it is what humans do when they think well at scale, made explicit and applied to a new collaborator.

Hutchins (1995) argued in *Cognition in the Wild* that cognitive work is not contained in heads. The plot of a ship’s position is computed by a

system: navigators, charts, plotting tables, instrument readings, written-down conventions. No one mind holds the whole computation. The chart is not a record of the navigator’s thinking — it is part of the thinking. Cognitive Cartography is a particular distribution: the artefact graph is part of the project’s cognition, not a passive record of it.

Scaife and Rogers (1996) sharpened the distinction between internal representations (mental models held in working memory) and external representations (graphics, tables, diagrams, documents). External representations radically alter what cognitive operations are possible: certain reasoning steps are easy on a diagram and hard in the head. The complexity of a project’s accumulated state is not a quantity to be carried internally; it is a structure to be externalised. Risko and Gilbert (2016) corroborate this empirically: people routinely offload to calendars, lists, sketches, GPS devices when cognitive demands rise, and the offloading demonstrably frees working-memory resources for harder operations.

The need rests on hard capacity limits. Baddeley (1992) established the architecture of working memory; Cowan (2001) placed the realistic capacity at roughly four chunks, sharpening Miller (1956)’s earlier estimate. Whatever the exact number, the order of magnitude is small. A project with eighty open considerations is not held in any one mind. It must live in the substrate.

Bartlett (1932) showed that memory is reconstructive: recall is shaped by pre-organised *schemata* that guide pattern-completion. An LLM asked “did we decide X?” performs an analogous reconstructive process — and like human memory, it can confabulate plausibly when the schema fits. The remedy is the same remedy human cognition has used for millennia: produce a stable external trace that can be consulted instead of reconstructed.

Marr (1982) formalised what software engineers and cognitive scientists have repeatedly rediscovered: complex information-processing problems decompose into distinct levels of analysis — computational, algorithmic, implementational. Each level has its own questions and its own appropriate language. Cognitive Cartography’s hierarchy of abstraction layers (Figure 2) is a Marr-shaped commitment. The brain itself, in current accounts (Friston, 2010; Clark, 2013), is a hierarchical predictive system; sustained inference is hierarchical because the flat alternative is intractable.



Star and Griesemer (1989) introduced *boundary objects*: artefacts that mean different things to different communities but enable coordination because they are robust enough to survive the variation. The artefacts of Cognitive Cartography are boundary objects between human and AI agent: each reads them with different prior contexts, but both can refer to the same artefact and update it without confusion. The stable identifier — KB-N, ADR-N, OQ-N — is the canonical boundary object.

Sweller (1988) formalised cognitive load theory: mental resources are spent on *intrinsic* (what the material demands), *extraneous* (what bad presentation imposes), and *germane* (what supports schema construction) load. Substrate discipline imposes intrinsic load (the discipline must be held in mind) but substantially reduces extraneous load (no more re-explaining what was decided) and increases germane load (the substrate makes patterns visible). Above a threshold of project complexity the trade is favourable; §7 addresses where that threshold lies.

### 3.2 Software engineering

The failure modes catalogued in §2 are not new; they have visited every team that has built complex software for more than a few months, and the tradition’s solutions have been recurrent enough to deserve names.

Nygaard (2011)’s Architectural Decision Records made the single most consequential of those solutions explicit. An ADR is a short append-only document recording one architectural decision, its context, its rationale, the alternatives considered, and its consequences. The crucial feature is the discipline of appending: an ADR is never edited; reversal happens via a new ADR that supersedes the old. The reasoning trail is preserved, not just the current answer. Fifteen years of practice across many organisations have shown the pattern resists rot in ways ad-hoc design notes do not. Cognitive Cartography adopts ADRs unmodified.

Cockburn (2005)’s hexagonal architecture (Ports and Adapters) makes a related move at the architectural level: the domain depends only on abstract *ports*; concrete adapters implement those ports. The discipline produces clean separability: the domain can be tested without touching infrastructure, and adapters can be swapped without disturbing the domain. The analogue at the substrate level is the separation between artefacts of different kinds, and the discipline of

cross-referencing only via stable identifiers.

Hunt and Thomas (1999) crystallised *Don’t Repeat Yourself* (DRY): every piece of knowledge must have a single, unambiguous, authoritative representation within a system. DRY is usually discussed at the level of code, but the principle generalises. In Cognitive Cartography, every fact has one home; everywhere else references it by id. Duplicate prose is treated as a bug — the same way duplicate code is. The duplicates will diverge, the divergence will be noticed late, and the cost of reconciling will exceed the cost of the original abstraction.

Candea and Fox (2003) proposed crash-only software: a program designed so that the only way to stop it is to crash, and the only way to start it is from a cold-start replay. The benefit is that the recovery path is the only path, and is therefore exercised on every restart rather than only in disasters. Cognitive Cartography adopts the same logic: every session begins by reading the artefact graph from disk, regardless of whether the previous session ended cleanly or crashed. There is no separate “warm restart” path.

These four practices — ADRs, hexagonal separation, DRY, crash-only — were developed independently for different software problems. Their convergence on a recognisably common shape — a disciplined, externalised, append-only, sparingly cross-referenced substrate that survives reorganisation — is itself evidence that the discipline is a recurrent solution. Cognitive Cartography names that shape and applies it where it has so far been least practised.

### 3.3 Agentic AI

The agentic-AI tradition is the youngest and supplies the contemporary motivation. AI systems built to operate over long horizons — to research, plan, browse, simulate — confront the substrate problem acutely: a context window is finite, the project is not.

Lewis et al. (2020) introduced retrieval-augmented generation: rather than packing all relevant context into the prompt, retrieve the relevant pieces from an external store at query time. RAG is now the dominant pattern for grounding LLM responses in specific knowledge bases. From the cognitive-cartography perspective, RAG is a partial answer: it solves “how to bring relevant context to the agent at the moment of need” but not “how to maintain that context’s coherence as the project evolves”. The substrate retrieved into

the agent’s working context is only as good as the substrate sitting at rest.

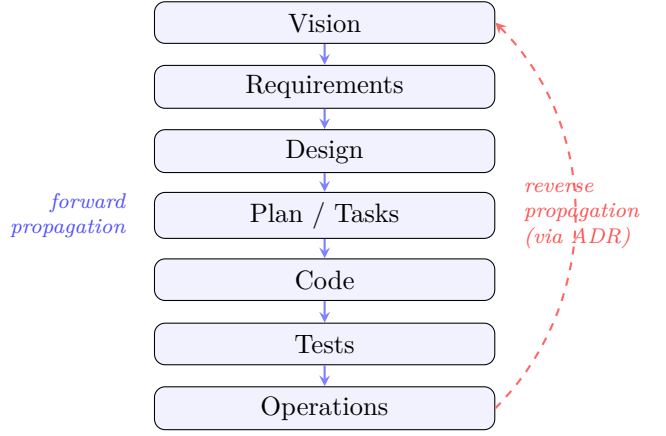
Packer et al. (2024) pushed harder, treating the LLM as the kernel of an operating system with explicit memory-management primitives — paging, hierarchical buffers, persistent storage. The MemGPT analogue of working memory and long-term memory mirrors the cognitive-science distinction between internal and external representations (Scaife and Rogers, 1996). Cognitive Cartography’s session warm-up is a coarse version of MemGPT-style paging, performed by a human practitioner at the start of work rather than continuously by the model.

Park et al. (2023) demonstrated that long-running simulated agents required explicit memory and reflection machinery to behave coherently across simulated days. They combined a flat memory store with retrieval and reflective summarisation. The simulated agents’ need for these mechanisms is the same need a practitioner has when collaborating with an LLM over many sessions, and the same set of solutions emerges in different idioms.

Shinn et al. (2023) showed that agents which reflect verbally on their own behaviour — generating critiques and revisions before retrying — outperform agents that do not. The analogue at the substrate level is the editing protocol: the discipline mandates a pre-edit reflection (READ, IDENTIFY SSOT, IMPACT-CHECK) before changes are made. Reflection is not optional; it is the protocol.

Bai et al. (2022) proposed Constitutional AI: encoding a small set of explicit rules the agent self-applies during generation. The constitution sits outside any single response and governs all of them. Cognitive Cartography’s edit protocol is a constitution at the methodological level — explicit rules a practitioner self-applies whenever touching the substrate.

The agentic-AI literature is converging from many directions on a common conclusion: long-horizon AI work requires explicit external state, retrieval and pre-fetch, reflection, and rules. Cognitive Cartography offers no new component over this literature; it offers a methodology that integrates them at the level the practitioner inhabits.



**Figure 2:** The hierarchy of maps. Each layer is a distinct representation; forward propagation cascades changes downward; reverse propagation requires deliberate revision via Decision Records.

## 4 Cognitive Cartography: The Discipline

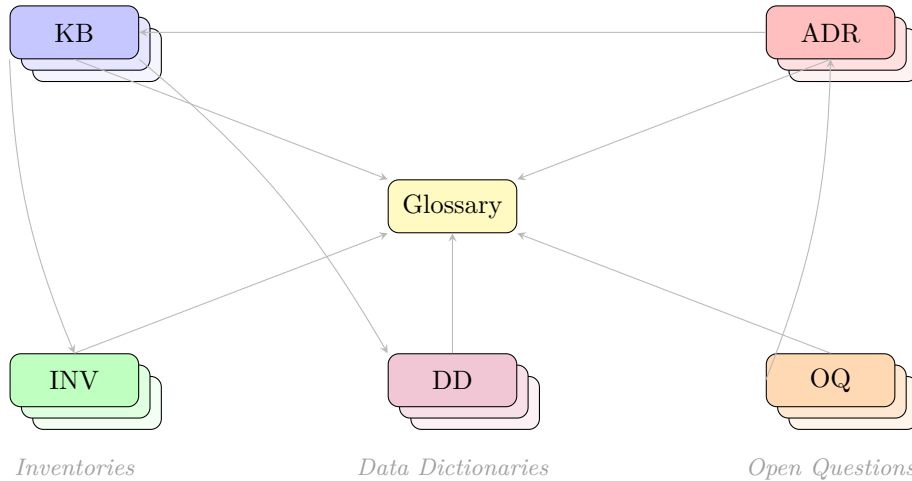
The discipline of Cognitive Cartography is specified by six artefact categories, a set of stable cross-referencing conventions, an explicit status lifecycle, an edit protocol, propagation rules across an explicit hierarchy of abstraction layers, an anti-pattern catalogue, and a session protocol. Each piece is modest in itself; the leverage comes from their combination (Figures 2, 3).

### 4.1 Six artefact categories

The substrate is composed of six kinds of artefact. Each kind has explicit semantics, an explicit format, and explicit rules about who is allowed to change it and when. The taxonomy is deliberately small: more categories would invite ambiguity about which one a given piece of content belongs to.

**Knowledge Bases (KB-N)** hold durable, slow-changing facts about the world the project operates in — the constraints of an external API, the regulatory environment, the assumptions of an underlying scientific tradition. KBs cite external sources with access dates and are reviewed at a cadence proportional to how fast their underlying world changes.

**Inventories (INV-N)** are exhaustive lists in well-defined categories: every supported order type, every emitted metric, every returnable error code. Where a KB explains, an Inventory enumerates. Inventories are designed to be machine-checkable: an automated test should be able to assert that every metric in the code is in the in-



**Figure 3:** The atlas. An artefact graph linking Knowledge Bases, Inventories, Data Dictionaries, Decision Records, and Open Questions, with the Glossary as central semantic anchor. Stacked boxes indicate that each category typically contains many artefacts; arrows are illustrative cross-references, not exhaustive. Cross-references use stable identifiers that survive reorganisation of the file tree.

ventory and every entry in the inventory is in the code.

**Data Dictionaries (DD-N)** are schemas: every field of every domain object, with type, semantics, invariants, and example. A Data Dictionary is the artefact a tool can validate against. It is also the artefact that makes interface boundaries explicit: two components agree on a Data Dictionary, and the dictionary becomes the contract.

**Architectural Decision Records (ADR-N)** record decisions, in the sense of Nygard (2011): a context, a decision, the alternatives considered, the consequences. ADRs are append-only. Reversal happens via a new ADR with a supersedes link to the old. The reasoning trail is preserved.

**Open Questions (OQ-N)** are the dual of decisions: deliberately unresolved questions, with target resolution dates, options under consideration, and resolution criteria. OQs prevent the silent slide from “we are deliberately not deciding this yet” into “we forgot we needed to decide this”. Once an OQ is resolved the resolution moves to an ADR; the OQ status updates to RESOLVED-BY-ADR-N; both records persist.

**Glossary** is the singular authority on terminology. Every term used twice in the substrate with the same meaning belongs in the glossary. If two artefacts disagree on a term’s meaning, the glossary wins or both are wrong; conversation is required. The glossary is the substrate’s semantic anchor.

The categories partition the substrate. Every

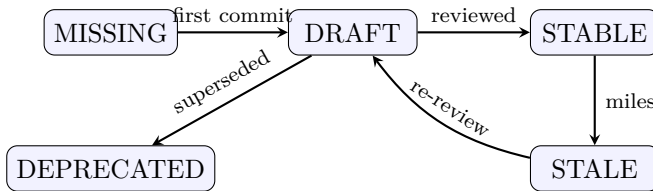
piece of project knowledge belongs to exactly one of the six. The categories also have characteristic update cadences — KBs change rarely, Inventories change with each new feature, ADRs are append-only, OQs accumulate and resolve, the Glossary grows and stabilises. The differential cadence is itself useful; it tells the practitioner what kind of artefact they are looking at without reading the content.

## 4.2 Stable identifiers and the cross-reference graph

Every artefact carries a stable identifier of the form <category-letter>-<number>: KB-1, INV-4, DD-7, ADR-12, OQ-23. Identifiers are assigned monotonically and never reused. Cross-references throughout the substrate use these identifiers — never file paths, never section numbers, never paraphrases.

The reason is simple. File paths reorganise; section numbers shift on insertion; paraphrases drift in meaning. Each is a route to reference rot. A stable identifier survives reorganisation: KB-3 remains KB-3 whether its file is at docs/kb/ or kb/ or has been moved into a subdirectory. The identifier-based cross-reference is a boundary object (Star and Griesemer, 1989) between today’s substrate layout and tomorrow’s: both layouts agree on what KB-3 is, even though they agree on nothing else about where it lives.

Every artefact also lists the artefacts it depends



**Figure 4:** Status lifecycle. Transitions are explicit and triggered; no artefact silently changes state.

on (`depends_on`) and the artefacts that reference it (`referenced_by`). Together these turn the substrate into a navigable graph. The graph supports impact analysis: before editing KB-3, the practitioner walks KB-3.`referenced_by` and reviews each downstream artefact. The graph supports orphan detection: an artefact whose `referenced_by` stays empty for more than one milestone is a candidate for retirement. The graph supports cycle detection: two artefacts that depend on each other suggest a missing third artefact that should own the shared content.

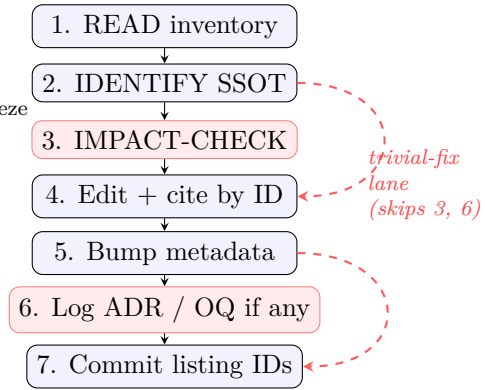
The atlas (Figure 3) is a partial picture of the graph at a moment in time. The full graph is implicit in the `depends_on` and `referenced_by` fields of every artefact, and could be rendered automatically by tooling (§8).

### 4.3 Status lifecycle

Every artefact has a status, drawn from five values (Figure 4): **MISSING** (named in the inventory but no file yet), **DRAFT** (file exists, content incomplete or not yet reviewed), **STABLE** (content current and complete to the level required), **STALE** (content was once stable but the underlying world has changed or an upstream artefact has changed since last review), and **DEPRECATED** (superseded; kept for history, must not be referenced by new work).

Each transition has an explicit trigger. **MISSING** to **DRAFT** is the first content commit. **DRAFT** to **STABLE** is owner review with completeness check. **STABLE** to **STALE** is either a milestone-freeze gate (if the artefact has not been reviewed by the gate, status auto-degrades) or a propagation event from an upstream artefact that changed substantively. **STALE** to **STABLE** is re-review. Anything to **DEPRECATED** is supersedure.

The discipline is that the status field is mandatory and accurate. “Eventually consistent” is treated as a bug, not a feature: if two artefacts disagree, the disagreement is logged with a target reconciliation date, not silently tolerated. The same applies between an artefact’s stated status



**Figure 5:** The edit protocol. The main path runs top-down through all seven steps. The trivial-fix lane (red dashed arrows) bypasses **IMPACT-CHECK** (step 3) and decision-logging (step 6), which are the steps that do not apply to typos and mechanical fixes. Steps 4, 5, and 7 always run.

and its actual freshness — a **STABLE** artefact with a six-month-old review date during a fast-moving project is not stable; it is stale, and should be labelled accordingly.

The status field, combined with the cadence rules from §4.1, gives the practitioner a quick estimate of the substrate’s overall health. A substrate with most artefacts in **STABLE** or **DRAFT** and few in **STALE** is in good shape. A substrate with many **STALE** or **MISSING** artefacts is signalling that the practitioner has out-grown the substrate or under-tended it.

### 4.4 The edit protocol

Every change to the substrate follows a sequence (Figure 5).

Before the edit: **READ** the inventory to confirm the artefact exists at the expected status; **IDENTIFY** which artefact owns the fact you intend to change (the Single Source of Truth, in the language of Hunt and Thomas (1999)); and **IMPACT-CHECK** by walking the artefact’s `referenced_by` list to identify downstream artefacts that may need to update.

During the edit: touch the minimum surface needed. Use stable identifiers in cross-references — never file paths, never paraphrases. Where you find yourself paraphrasing content from another artefact, replace the paraphrase with a citation. Where you find that the same fact lives in two artefacts, consolidate.

After the edit: bump `last_reviewed`; if the change is substantive, bump `version`; if status changes, update the inventory in the same commit. If the edit reflects a decision among alternatives,



write the corresponding ADR in the same commit. If the edit raises a question that cannot be resolved in line, write the corresponding OQ. Commit messages list every artefact touched, by stable identifier.

**The trivial-fix lane.** A protocol that imposes the full sequence on every typo is a protocol that gets bypassed. A trivial-fix lane is built in: typos, formatting, broken markdown, link fixes that do not change meaning skip the IMPACT-CHECK and the decision-logging steps. The `last_reviewed` field still bumps. If you are uncertain whether a fix is trivial, it isn't; use the full lane. The trivial lane exists to reduce extraneous cognitive load (Sweller, 1988), not to launder ceremony around non-trivial changes.

## 4.5 Forward and reverse propagation

Changes at one layer of the abstraction hierarchy (Figure 2) may imply work at others. The discipline distinguishes two propagation directions.

**Forward propagation** is the high-to-low cascade. A change to Requirements may flag downstream review of Design, Code, Tests, and Operations. The discipline mandates that forward propagation happen in the same commit if at all possible, or be filed as an explicit OQ with a target resolution date if it cannot. A lurking “I’ll update the tests later” is the road to drift.

**Reverse propagation** is the low-to-high direction. Implementation may reveal a constraint not anticipated in design; a test may surface a missing requirement; operations may reveal a runtime constraint that requires architectural revision. The discipline mandates that reverse propagation always pass through an ADR. A higher-layer artefact is never silently retro-fitted to match what was built; the ADR records that a lower-layer discovery has caused a higher-layer revision.

A higher-layer artefact may be *frozen* for a development phase — typically Requirements is frozen during implementation. A frozen artefact is not edited; an ADR proposing the unfreeze is required first. The protocol is heavy by design: opening Requirements during implementation is exactly the kind of action that benefits from deliberate friction.

The discipline of bidirectional propagation is what keeps the hierarchy of abstraction layers more than a label. The layers mean what they say only because changes to any layer cause review of the others, and the reviews leave traces.

## 4.6 Anti-patterns

Most damaging mistakes recur. Naming them and forbidding them explicitly is cheap insurance. The catalogue below is the discipline’s “do not”.

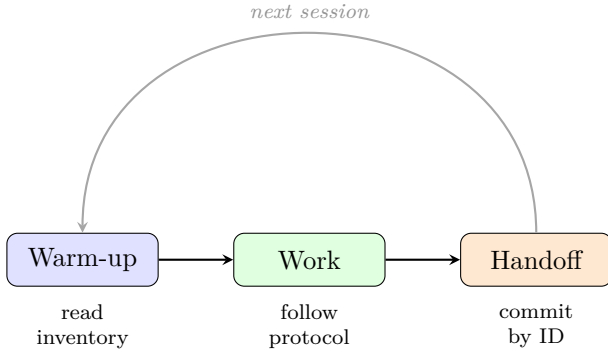
1. Editing a STABLE artefact without bumping `last_reviewed`. The change is invisible to the next reviewer.
2. Pasting the same content in two artefacts. A fact has one home.
3. Making a decision in conversation without writing the ADR. Phantom decisions are how this fails most.
4. Renaming an artefact’s file without sweeping cross-references — prevented by the stable-identifier convention if the identifier remains.
5. Creating an artefact without adding a row to the inventory. The artefact becomes an orphan no one will discover.
6. Quoting another artefact’s prose verbatim. The duplicate will drift; the right move is a citation.
7. Letting Open Questions accumulate without target resolution dates. The OQ register becomes a wishlist instead of a register.
8. Mixing levels — design specifics in Requirements, file paths in Design, configuration values in the Glossary. Each level has its language.
9. “I’ll update the tests later.” The deferred propagation becomes the next session’s drift.
10. Editing the body of a past ADR. Reversal goes through a superseding ADR; the past ADR is annotated with its supersession but otherwise immutable.
11. Bypassing the SSOT rule “just for clarity”. Clarity is achieved through structure, not duplication.

The catalogue is not exhaustive. New anti-patterns surface as the discipline is practised in new domains; the catalogue grows append-only.

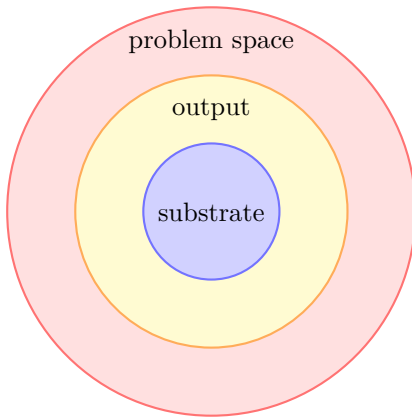
## 4.7 Session protocol

The session-level discipline ensures every session begins by reading the substrate and ends by depositing state back into it (Figure 6). The substrate *shadows* the problem space at lower fidelity than full understanding but with substantially higher stability (Figure 7).

**Warm-up.** Before any productive work in a new session: read the inventory; read the protocol’s TL;DR refresher; read the artefacts the current task touches; spawn supplementary reads (or explorer agents) for any unfamiliar dependen-



**Figure 6:** Session lifecycle. Every session takes the same warm-up path, whether the prior session ended cleanly or crashed. The *next session* loop is the same loop on every restart.



**Figure 7:** Substrate coverage. The substrate (inner disc, blue) shadows the output ring (yellow) and the problem space (outer red) at lower fidelity than full understanding, but with substantially higher stability. Without a substrate, the output is fragile against drift.

cies. Five to ten minutes spent on warm-up substantially reduces the rate of phantom decisions, reference rot, and decided-versus-considered confusion later in the session.

**Work.** Apply the edit protocol (§4.4) to every change. Cite artefacts by stable identifier in conversation, not by paraphrase. Use task-tracking primitives to make multi-artefact edits explicit and atomic. When in doubt about which artefact owns a fact, stop and confirm before editing.

**Handoff.** Before declaring the session done: every edit is committed; every new MISSING artefact created has a stub frontmatter and an inventory row; every decision has its ADR; every raised question has its OQ; `last_reviewed` and `version` are current on touched artefacts; the inventory’s priority queue reflects the new state. The next session’s warm-up is the previous session’s handoff, read by whoever (or whatever) is

doing the next session.

The protocol applies symmetrically across the human and AI collaborators. Both read the substrate at session start. Both update it during work. Both leave it consistent at session end. The asymmetry — that the AI agent’s session is bounded by its context window in a way the human’s is not — is precisely what the discipline is designed to compensate for.

## 5 The Synthesis: Discipline ↔ Cognitive Principle ↔ Failure Mode

This section is the paper’s central synthesis. Each element of Cognitive Cartography from §4 maps to a cognitive principle from §3 that grounds it and to a failure mode from §2 that it mitigates. Table 1 lays out the mapping.

The shape of the table matters as much as its content. Every discipline element has a cognitive precedent, and every discipline element addresses an observable failure. The discipline is not arbitrary, and it is not a list of best practices. It is a small set of structural choices, each with two sources of justification: why it should work (the cognitive principle) and what happens when it is absent (the failure mode).

Two readings of the table are useful. From left to right, it answers “why these elements?”. From right to left, it answers “what to use against this failure?”. Both readings produce the same set of correspondences. The redundancy is the synthesis’s robustness: every element is on the table for two independent reasons.

A reader pressed for time should read §1 and Table 1. The argument of the paper survives the reduction.

## 6 A Worked Example

We illustrate Cognitive Cartography with a single multi-month project. Specifics are generalised: the project was a live algorithmic-execution engine bridging a research backtesting framework to a regulated broker. The engineering content of the project is not the point. The point is the substrate trajectory.

The project began as a chat. A practitioner described the problem: a backtesting framework that produces strategies, a broker API to send them to live, the gap between the two. The first

| Discipline element                                  | Cognitive principle                                                                        | Failure mode addressed                         |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------|
| Hierarchy of abstraction layers                     | Marr’s three levels of analysis (Marr, 1982); hierarchical predictive coding (Clark, 2013) | Drift across iterations; lost intent           |
| Single Source of Truth + DRY-for-prose              | External representations (Scaife and Rogers, 1996); cognitive load (Sweller, 1988)         | Decided-vs-considered confusion; reference rot |
| Stable identifiers (KB-N, ADR-N, OQ-N)              | Boundary objects (Star and Griesemer, 1989); persistent symbolic anchors                   | Reference rot; phantom decisions               |
| Append-only Decision Records                        | Distributed cognition (Hutchins, 1995); preservation of reasoning trail                    | Phantom decisions; silent reversal             |
| Status lifecycle (MISSING → DRAFT → STABLE → STALE) | Schema theory (Bartlett, 1932); affordances of state                                       | Drift; stale knowledge                         |
| Edit protocol with trivial-fix lane                 | Cognitive load theory (Sweller, 1988); chunking (Miller, 1956; Cowan, 2001)                | Restart cost; over-ceremony abandonment        |
| Glossary as authoritative semantic anchor           | Encoding specificity (Tulving and Thomson, 1973)                                           | Terminology drift                              |
| Session warm-up + handoff protocol                  | Cognitive offloading (Risko and Gilbert, 2016)                                             | Restart cost; context-window saturation        |

**Table 1:** The synthesis. Each discipline element, the cognitive principle that grounds it, and the failure mode (§2) it addresses.

artefact deposited to the substrate was a Requirements document — flat, around six thousand words, the practitioner’s first pass at what would be built and why.

The second artefact was a Context Inventory, listing the categories of substrate the project would need: Knowledge Bases, Inventories, Data Dictionaries, Decision Records, Open Questions, Glossary. The inventory was at first largely MISSING; it named the artefacts that would be needed without yet writing any.

The third artefact was a Context Protocol — the discipline document specifying how the substrate would be edited, how cross-references would work, what the status lifecycle was. The protocol came before most of the content because it determined how the content would be added.

From there the substrate grew. The first Knowledge Base captured the strategy taxonomy — the archetypes the project would support. That KB raised ten Open Questions about scope. The practitioner made decisions on those OQs in a single working session; the decisions became the project’s first set of Architectural Decision Records. The KB updated to mark the OQs as RESOLVED-BY-ADR; the OQs were not deleted, just re-statused.

Knowledge Bases on the broker’s API capabilities and pacing limits followed. A Cost Dictionary on the project’s cost models. A Companion

Projects map clarifying boundaries with related work. A Frameworks Survey reviewing existing solutions and recording which were adopted, studied, or rejected — and why. Within a few working sessions, thirteen Knowledge Bases existed; three Inventories; nineteen Architectural Decision Records; twenty-two Open Questions; one Glossary that had absorbed the terminology sections of multiple other artefacts.

The Requirements document, originally a flat draft, underwent a v0.2 pass: sections that had been narrative summaries of survey content, decision rationale, and open questions were collapsed into stable-identifier references to the relevant Knowledge Bases, Architectural Decision Records, and Open Questions. The document became substantially shorter, more authoritative, and easier to update — because the content it had been carrying now lived in artefacts whose discipline was to be authoritative.

The substrate trajectory is the point. The project did not begin with a plan and execute it. It began with the cheapest possible substrate (a Requirements document) and grew the substrate as the work demanded it, with the discipline ensuring that every growth step left the artefact graph coherent. By the time implementation began, the substrate was sufficient to absorb a session’s worth of work without drift, and the practitioner could resume after a break of weeks

without reconstruction cost.

The example does not prove the discipline works. It shows what the discipline looks like in flight on a non-trivial project. The metrics — thirteen KBs, nineteen ADRs, twenty-two OQs — are incidental. The trajectory — substrate as living artefact, growing to match the project’s demand for stability — is the substance.

## 7 The Practice in the Hand

Cognitive Cartography is not free. The discipline costs effort, and the effort is borne by the practitioner. This section names those costs honestly.

The protocol’s heaviest cost is at the start. The first substrate-disciplined session on a new project takes longer than the same session run as flat chat, because the inventory is MISSING and must be created, the protocol must be set up, and the practitioner is doing two things at once: working on the project, and building the substrate the project will live in. This investment pays back roughly from session three or four, when the substrate begins to absorb context that flat chat would otherwise have demanded re-establish each session.

The ongoing cost is intrinsic load (Sweller, 1988): the discipline must be held in mind. The pre-edit READ, the IDENTIFY SSOT, the IMPACT-CHECK — none of these is hard, but together they impose a small overhead on every change. The trivial-fix lane absorbs much of the overhead for small changes; the rest is amortised across the changes the discipline saves elsewhere.

There is also a hidden requirement: someone must hold the substrate in mind even while distributing it. The practitioner who maintains a substrate at the level of discipline this paper specifies has spent years building that capability across many projects, in domains both AI-related and not. The discipline articulated here is not Monday’s lesson; it is the externalisation of years of internal habit.

The threshold above which Cognitive Cartography pays back is roughly: the project will span more than a handful of sessions, or more than a handful of decisions of consequence, or more than one collaborator. Below that threshold, flat chat and unstructured notes are appropriate. Above it, every additional unit of complexity exacerbates the failure modes of §2 and therefore widens the margin by which the discipline pays.

The discipline scales down naturally. Not every

project needs thirteen Knowledge Bases; some need three. Not every protocol clause is invoked on every change; the trivial-fix lane is built for the common case. The aim is not to maximise discipline; it is to match discipline to project, with the project’s complexity as the dial.

## 8 What Tooling Could Do for Cartographers

The discipline is practicable today with no special tooling — a text editor, a version-control system, and a project structure suffice. But the discipline could be made substantially less expensive with modest tooling support. We sketch the possibilities here without proposing a system.

**Frontmatter validators.** An artefact’s frontmatter (`id`, `title`, `status`, `owner`, `last_reviewed`, `version`, `depends_on`, `referenced_by`) is structured data. A pre-commit hook can check that every artefact has frontmatter, that the schema is correct, and that `last_reviewed` is more recent than the file’s last modification time.

**Stable-identifier link checkers.** Every cross-reference is of the form KB-N, ADR-N, OQ-N. A linter can confirm every reference resolves to an existing artefact whose frontmatter `id` matches.

**Inventory consistency.** The inventory of artefacts is itself an artefact; an automated check should verify that every file in the document tree has a row in the inventory, and every row in the inventory has a file.

**Status-lifecycle automation.** A milestone-freeze gate should be able to walk every STABLE artefact and check its `last_reviewed` date against the gate; artefacts that haven’t been reviewed by the gate should auto-degrade to STALE. The reverse — promotion to STABLE — remains explicit, but degradation can be safely automatic.

**Session warm-up bundlers.** A tool that, given the inventory and the current task, produces a recommended reading list for session warm-up — the artefacts in the task’s `depends_on` closure, ordered by how recently each changed. The human still reads; the tool just curates.

**Cross-reference graph visualisation.** The `depends_on` / `referenced_by` graph is a navigable structure. A static visualiser that produces a current picture of the atlas (Figure 3) is mechanical and would replace the hand-drawn diagram.

None of this is novel; all of it is specific application of existing patterns (linters, schema val-



icators, graph visualisation). What is missing is a system that knows about substrate as a first-class concept and supports the patterns together. We do not build such a system here. We note that the discipline does not require it. The system would be a force multiplier; the discipline operates without it.

## 9 Honest Costs and Limits

A self-respecting methodology paper acknowledges where the methodology fails. Cognitive Cartography is not universally helpful. This section names where it doesn't help, where it costs more than it returns, and where the assumptions break.

**Truly novel research.** The discipline assumes the project's hierarchy of abstractions can be articulated. In genuinely novel research — where the right ontology is not known until the work is mostly done — the substrate cannot be designed in advance, and forcing one is harmful. The discipline can be applied retroactively, after the right structure has emerged, but it cannot accelerate the emergence. For exploratory work, flat notes are often better; substrate discipline applies once the work crosses into elaboration.

**Bureaucracy risk.** A discipline whose ceremony exceeds its benefit is a discipline that gets ignored or performatively followed. The trivial-fix lane is built specifically to keep this risk manageable, but the lane only works if it is generously sized. A practitioner who narrows the trivial lane until every change requires the full sequence will find their team — or their agent collaborator — quietly bypassing the discipline. The discipline must scale down or it scales out of relevance.

**Substrate-author expertise as hidden requirement.** A substrate is no better than the discipline of its author. The discipline articulated in this paper looks straightforward on the page; in practice it requires judgement honed across many projects. A practitioner attempting Cognitive Cartography for the first time will produce a worse substrate than the same practitioner three projects later. We have no way to shortcut this: the discipline is teachable, but the judgement is practised.

**Multi-author coordination.** The discipline as specified here assumes a single primary substrate author, with collaborators (human and AI) operating against the substrate. Multiple authors editing the substrate concurrently introduce co-

ordination problems — branch-merge semantics, conflicting status changes, duplicate ADRs — that this paper does not address. The discipline extends to multi-author settings, but the extension is its own work.

**The discipline is not a thinking aid.** Cognitive Cartography helps a practitioner who has something to say organise and sustain it. It does not generate ideas, evaluate alternatives, or verify correctness. A well-organised wrong project is still wrong. The discipline preserves the practitioner's reasoning; it does not produce it.

**Model capability still matters.** We have argued that substrate matters as much as model. We have not argued that model capability is irrelevant. A model that cannot follow instructions, cite stable identifiers, or refrain from hallucinating in spite of contradicting evidence cannot be a useful collaborator on a disciplined substrate. Substrate amplifies model capability; it does not replace it.

These caveats do not invalidate the discipline. They locate it. Cognitive Cartography is the right tool for sustained complex work with a capable AI collaborator, above a complexity threshold, where the project's structure is articulable. Outside that envelope it costs more than it returns, and we do not recommend it.

## 10 A Cognitive Practice for the AI Era

We close on the deepest implication.

AI raises the importance of work humans have always needed to do. Externalising thought, structuring it hierarchically, preserving reasoning trails, naming things stably — none of this is new. Books, libraries, encyclopedias, scientific notation, version control, lab notebooks, architectural decision records: every generation has produced apparatus for stabilising thought across time and distributing it across minds. The apparatus has worked.

What is new is the collaborator. A large language model is the first collaborator with the cognitive capacity of a careful specialist and the persistent memory of a goldfish. The asymmetry is unique: any session begins with the agent's full intelligence intact and the project's accumulated context absent. Substrate is how the absence is repaired, every session, indefinitely.

The discipline that addresses this is not invented for it. The discipline is recovered, from

cognitive science, from software-engineering, and from agentic-AI literatures, and made explicit. The practice is fundamentally human: a person cultivates it, with AI as participant. AI’s limitations motivate it; human practice executes it. The practice survives across model generations because it is about how minds collaborate on complex problems, not about which model is current.

Substrate is the leverage point currently most under-practised in AI work. Model capability per token will continue to grow, and context windows will continue to lengthen, and benchmarks will continue to shift. None of this changes the substrate problem. The complexity of any project that matters grows faster than any plausible future model can absorb in a single context. The projects that succeed will be the ones whose substrate keeps pace.

Cognitive Cartography is one form of the practice. There may be others. What matters is that the practice exists, that it is explicit, that it is taught, and that it is shared. We have described one form. We expect the form to evolve. The form’s evolution is itself a forward-propagation event the discipline foresees.

## Acknowledgements

The discipline articulated in this paper is the author’s, cultivated across many disciplines and projects over years of sustained complex work, of which collaboration with current-generation AI agents is the most recent and most demanding instance — not its origin. The discipline’s choices — its artefact categories, the status lifecycle, the edit protocol, the propagation rules, and the discipline’s overall shape — were practised and refined long before they were named here. Anthropic’s Claude provided substantial assistance in drafting and stress-testing the articulation in this paper, particularly in tracing precedents in the cognitive-science and software-engineering literatures and in helping render implicit practice into explicit specification. The writing was a collaboration in the spirit the paper itself describes; the discipline it articulates was not.

## References

- Alan Baddeley. Working memory. *Science*, 255 (5044):556–559, 1992.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Frederic C. Bartlett. *Remembering: A Study in Experimental and Social Psychology*. Cambridge University Press, 1932.
- George Candea and Armando Fox. Crash-only software. In *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, 2003.
- Andy Clark. Whatever next? predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36(3): 181–204, 2013.
- Alistair Cockburn. Hexagonal architecture (ports and adapters), 2005. Online.
- Nelson Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.
- Karl Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.
- Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.
- Edwin Hutchins. *Cognition in the Wild*. MIT Press, Cambridge, MA, 1995.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- David Marr. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman, 1982.

George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.

Michael Nygard. Documenting architecture decisions, 2011. Online; see <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2024.

Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Evan F. Risko and Sam J. Gilbert. Cognitive offloading. *Trends in Cognitive Sciences*, 20(9):676–688, 2016.

Mike Scaife and Yvonne Rogers. External cognition: How do graphical representations work? *International Journal of Human-Computer Studies*, 45(2):185–213, 1996.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Susan Leigh Star and James R. Griesemer. Institutional ecology, “translations” and boundary objects: Amateurs and professionals in Berkeley’s museum of vertebrate zoology, 1907–39. *Social Studies of Science*, 19(3):387–420, 1989.

John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

Endel Tulving and Donald M. Thomson. Encoding specificity and retrieval processes in episodic memory. *Psychological Review*, 80(5):352–373, 1973.

## A The Cartographer’s Compact Field Manual

This appendix collects the discipline in a form a practitioner can print and pin. Repetition is by design: the protocol’s steps are short enough that the appendix should be readable on its own.

### Frontmatter (every artefact)

```
---
id: KB-NNN | INV-NNN | DD-NNN
    | ADR-NNN | OQ-NNN
title: human-readable title
status: MISSING | DRAFT | STABLE
    | STALE | DEPRECATED
owner: Name | shared
last_reviewed: YYYY-MM-DD
version: 0.1
sources: [URLs with date accessed]
depends_on: [ID, ID, ...]
referenced_by: [ID, ID, ...]
---
```

### Edit protocol (every non-trivial change)

1. READ the inventory; locate the artefact.
2. READ its frontmatter.
3. IDENTIFY the SSOT for the fact you intend to change.
4. IMPACT-CHECK by walking **referenced\_by**.
5. Decide the lane: trivial / normal / decision-bearing.
6. Edit; cite by stable identifier; do not restate.
7. Bump **last\_reviewed**; bump **version** if substantive.
8. Update inventory if status changed.
9. Log decision as ADR if applicable.
10. Log question as OQ if applicable.
11. Commit; commit message lists every artefact touched, by ID.

### Trivial-fix lane

For typos, formatting, broken links that do not change meaning: skip steps 4, 9, 10. Still bump **last\_reviewed**. If unsure whether a fix is trivial, it isn’t; use the full lane.

### Status transitions

- MISSING → DRAFT: first content commit.
- DRAFT → STABLE: owner review; complete; no TODO.

- STABLE  $\rightarrow$  STALE: `last_reviewed` older than current milestone freeze, or upstream changed.
- STALE  $\rightarrow$  STABLE: re-review.
- Any  $\rightarrow$  DEPRECATED: superseded.

## Anti-patterns (do not)

1. Edit STABLE without bumping `last_reviewed`.
2. Paste the same content in two artefacts.
3. Decide in conversation without an ADR.
4. Create an artefact without an inventory row.
5. Quote another artefact's prose verbatim.
6. Let OQs accumulate without target dates.
7. Mix levels of abstraction in one artefact.
8. Defer propagation ("I'll update tests later").
9. Edit a past ADR's body.
10. Bypass SSOT "just for clarity".

## Session protocol

**Warm-up.** Read inventory; read protocol TL;DR; read task's artefacts; spawn supplementary reads.

**Work.** Apply edit protocol; cite by ID; track multi-artefact edits as a unit.

**Handoff.** Every edit committed; every new artefact has frontmatter and inventory row; decisions in ADRs; questions in OQs; metadata current; inventory's priority queue reflects new state.